

MAS Final Project: Detailed Workflows and Execution Playbook

1. Scope and Goal

This chapter provides the complete operational workflow for the project, from startup to planning, simulation, verification, refinement, and reporting.

It is written as an execution playbook tied directly to current code behavior.

Covered modules in workflow context:

- `main.py`
- `config.py` / `config.yaml`
- `env.py`
- `agent.py`
- `pathfinding.py`
- `symmetry_reduction.py`
- `verification.py`
- `refinement.py`

2. System Lifecycle at a Glance

One full end-to-end cycle:

1. Load defaults and optional YAML config.
2. Apply CLI overrides.
3. Initialize environment and planner.
4. Run selected mode:
 - interactive,
 - simulate,
 - eval,
 - detect-symmetry,
 - verify-refine.
5. During runtime:
 - planner computes actions,
 - environment executes and records conflicts/metrics,
 - optional verification uses quotient states,
 - optional refinement injects constraints.
6. Log and inspect outputs (collisions, safety margin, constraints, debug fields).

3. Boot Workflow (Accurate Startup Sequence)

`main.py` startup order:

1. Build parser via `_build_parser()`.
2. Parse CLI args.
3. Initialize logging at INFO.

4. Attempt `cfg.load_from_yaml(args.config)`.
5. Resolve unset optional runtime args from config via `_apply_defaults(args)`.
6. If `--seed` provided, call `set_seed(seed)`.
7. If `--cell-size` provided, overwrite `cfg.CELL_SIZE`.
8. Route execution:
 - `--detect-symmetry` branch first,
 - `--verify-refine` branch second,
 - else route by `--mode`.

Operational implication:

- `--detect-symmetry` and `--verify-refine` bypass normal `--mode` runtime.

4. Configuration Workflow

Parameter source precedence:

1. hardcoded defaults in `config.py`,
2. optional `config.yaml`,
3. explicit CLI values.

Practical workflow:

1. Keep project-wide baseline in `config.yaml`.
2. Override experimental knobs with CLI flags per run.
3. Log seed and critical arguments with every experiment for reproducibility.

Recommended reproducibility tuple to record:

- seed,
- horizon/trials,
- min_separation,
- refine_iterations/refine_max_constraints,
- include_shelves flag,
- agent/shelf/grid sizes.

5. Interactive Workflow (Live Runtime)

Command:

```
python main.py --mode interactive --render --seed 42
```

Execution timeline:

1. Create `WarehouseEnv(render=True)`.
2. Create `CooperativePlanner(...)`.
3. Loop per frame:
 - consume UI events via `env.handle_event`,
 - `planner.compute_actions(env)`,
 - `env.step(actions)`,

- auto-reset if done,
- `env.render()`.

Useful for:

- visual sanity checks,
- congestion behavior inspection,
- orbit coloring/role visualization,
- conflict overlay inspection.

Exit controls:

- close window or press `q`.

6. Simulation Workflow (Batch Episodes)

Command:

```
python main.py --mode simulate --episodes 20 --steps-per-episode 200 --log-interval 1 --seed 42
```

Execution timeline:

1. Initialize env/planner once.
2. For each episode:
 - reset env,
 - step until done or max steps.
3. Aggregate collisions globally.
4. Log average collisions per agent per episode periodically.

When to use:

- quick robustness checks,
- planner tuning loops,
- collision trend monitoring across episodes.

7. Evaluation Workflow (Metric-Focused)

Command:

```
python main.py --mode eval --eval-episodes 50 --steps-per-episode 200 --seed 42
```

Execution timeline:

1. Create no-render env and planner.
2. Call `env.evaluate(...)`.
3. Return one summary metric:
 - avg collisions per agent per episode.

When to use:

- comparing planner configurations,
- before/after refinement comparisons,
- reporting headline operational safety quality.

8. Symmetry Inspection Workflow

Command:

```
python main.py --detect-symmetry --seed 42
```

Execution timeline:

1. Create env and reset.
2. Call `build_quotient_model(env)`:
 - detect role orbits,
 - build mapping and representatives.
3. Print:
 - each orbit list,
 - full quotient summary dict.

What this confirms:

- role grouping behavior at initial state.

What it does not confirm:

- long-horizon quotient compression rate during runtime.

9. Verify-Refine Workflow (Core Safety Loop)

Baseline command:

```
python main.py --verify-refine --verify-horizon 30 --verify-trials 20 --refine-iterations 3
```

Shelf-aware variant:

```
python main.py --verify-refine --verify-include-shelves --verify-horizon 30 --verify-trials
```

Iteration-level sequence:

1. Run `verify_on_quotient(...)`.
2. If safe:
 - log success and `delta_q`,
 - log average collision rate and current constraints,
 - stop loop.
3. If unsafe:
 - extract `conflicts` and `counterexample`,
 - call `refine_planner_with_conflicts(...)`,
 - inject constraints into same planner instance,
 - continue next iteration.
4. If iteration cap reached and still unsafe:
 - log exhaustion warning and summary.

Core concept:

- planning is gradually constrained by observed failures.

10. Single-Step Runtime Workflow (Planner + Environment)

For each simulation timestep:

1. `planner.compute_actions(env)`:
 - update idle/assignments,
 - compute planning order,
 - plan each robot with reservations and constraints,
 - sanitize immediate conflicts,
 - return integer action list.
2. `env.step(actions)`:
 - decode actions,
 - record intent conflicts,
 - apply turn/pick/drop and forward intents,
 - resolve conflicts and apply valid movement,
 - compute rewards and metrics,
 - update conflict/event/heat telemetry,
 - return next states and collision count.

Data produced each step (important artifacts):

- action vector,
- rewards vector,
- collisions count,
- `env.last_conflicts`,
- `env._planner_debug_by_agent`,
- heatmaps and event log updates.

11. Safety Artifact Workflow

11.1 Conflict Artifact Production

Produced by environment:

- boundary conflict,
- vertex conflict,
- edge conflict.

Stored in:

- `env.last_conflicts`

11.2 Verification Artifact Production

Produced by verifier when unsafe:

- `counterexample` trace,
- `actions` history,
- `conflicts` list,

- `delta_q`,
- `avg_collision_rate`.

11.3 Refinement Artifact Consumption

Consumed by refiner:

- conflict dicts + trace.

Result:

- planner constraint table mutation,
- reported `applied_constraints`.

12. Quotient Verification Workflow (Detailed)

Per trial in verifier:

1. Reset environment.
2. Check initial pairwise separation.
3. Loop `step_idx` up to horizon:
 - compute canonical key with `canonicalize_state(...)`,
 - if key revisited: break this trial early (cycle),
 - else execute one planning+step transition,
 - update margin `delta_q`,
 - stop and return unsafe on collision/conflict/separation violation.

End of all trials:

- if no unsafe event: return safe with summary metrics.

Interpretation guidance:

- safe result means bounded-trial safe under sampled resets and current parameters,
- not a global/unbounded formal proof.

13. Constraint Evolution Workflow

Planner constraints are cumulative across verify-refine iterations (same planner instance).

Added constraints live in:

- `planner.constraints.positions`
- `planner.constraints.edges`

Logged by:

- `_log_refine_summary(...)` in `main.py`.

Practical effect:

- search space narrows over iterations,
- repeated unsafe patterns should become harder/impossible.

Risk:

- excessive constraints can over-restrict and increase waiting/deadlock pressure.

14. Assignment and Target Workflow

At planner step:

1. `AssignmentManager` matches free robots to requested shelves via Hungarian matching.
2. Carrying requested robots get delivery targets via goal-slot matching.
3. Planner sets `_planner_allowed_shelf_entries` for assigned shelf entry control.

At agent move check:

- `_occupied_by_shelf` only allows assigned requested shelf entry for pickup.

Operational consequence:

- pickup ownership is planner-governed, not opportunistic.

15. Deadlock Mitigation Workflow

Combined mechanisms:

1. rotating planning priority (`priority_offset`),
2. idle tracking with assignment release (`IdleTracker`),
3. fallback immediate escape action when A* path stalls,
4. final immediate conflict sanitization.

Observed expected behavior:

- robots may wait under congestion,
- prolonged inactivity triggers escape/reassignment behavior.

16. Debugging Workflow (Field-by-Field)

Use `env._planner_debug_by_agent` after each step.

Key fields to inspect:

- `mode` (decision path taken),
- `target`,
- `astar_found`,
- `path_len`, `path_preview`,
- `blocked*_count`,

- `chosen_action_pre_sanitise`,
- `chosen_action`,
- `sanitized`.

Typical diagnosis patterns:

1. `mode=no_spacetime_path_wait` repeatedly:
 - likely over-constrained congestion/horizon cap.
2. frequent `sanitized=True`:
 - local first-step conflicts are common.
3. high `blocked_static_count`:
 - shelf geometry heavily restricting passage.

17. Experiment Workflow (Recommended Protocol)

Phase A: Baseline

1. Run eval with fixed seed and baseline params.
2. Record collision metric.

Phase B: Verify

1. Run verify-refine with `refine_iterations=1` (verify only effect snapshot).
2. Record safe flag, `delta_q`, avg collision rate.

Phase C: Refine Loop

1. Run full verify-refine with multiple iterations.
2. Record per-iteration:
 - safe/unsafe,
 - `delta_q`,
 - `applied_constraints`,
 - final constraint counts.

Phase D: Post-Refine Eval

1. Evaluate refined planner in simulation/eval mode.
2. Compare against baseline metrics.

Phase E: Ablation

1. Repeat with `--verify-include-shelves`.
2. Repeat with modified `PLAN_HORIZON` / `ASTAR_MAX_NODES`.
3. Compare tradeoff between safety and throughput.

18. Command Playbook (Operational Set)

Install:


```
pip install -r requirements.txt
```

Interactive:

```
python main.py --mode interactive --render --seed 42
```

Simulation:

```
python main.py --mode simulate --episodes 20 --steps-per-episode 200 --seed 42
```

Evaluation:

```
python main.py --mode eval --eval-episodes 50 --steps-per-episode 200 --seed 42
```

Symmetry detection:

```
python main.py --detect-symmetry --seed 42
```

Verify-refine:

```
python main.py --verify-refine --verify-horizon 30 --verify-trials 20 --refine-iterations 3
```

Verify-refine with shelf-aware quotient:

```
python main.py --verify-refine --verify-include-shelves --verify-horizon 30 --verify-trials
```

19. Output Interpretation Workflow

If `safe=True` quickly

- current planner+config is robust for bounded test budget.
- still run more seeds/trials before strong claims.

If repeatedly unsafe

1. inspect conflict types and locations,
2. inspect applied constraints count,
3. increase trials/horizon for richer failure evidence,
4. adjust planner caps/horizon to reduce deadlock-induced unsafe events.

If collision rate improves but still unsafe

- refinement is helping but insufficient,
- raise `refine_iterations` or improve conflict translation logic.

20. Common Failure Scenarios and Handling Workflow

1. Config parse warnings:
 - validate YAML structure and key types.
2. No meaningful refinement progress:
 - inspect whether conflicts are mostly "`separation`" (currently ignored by refiner).
3. Over-constrained planner:

- large constraints with persistent waits; reduce refine aggressiveness or tune planner search budget.
4. Runtime slowdown:
- reduce trials/horizon/rendering or adjust A* caps.

21. Accuracy Notes and Limits

This workflow is accurate to current code behavior:

- bounded verification,
- role-based symmetry reduction,
- conflict-driven hard constraints,
- deterministic planner with heuristic escape/sanitization layers.

Not guaranteed by current code:

- unbounded safety proof,
- global optimality of plans,
- monotonic improvement across every refinement iteration.

22. End-to-End Narrative (Presentation-Ready)

The operational story is:

1. initialize deterministic warehouse world,
2. run cooperative planner that handles assignment, space-time routing, and conflict avoidance,
3. compress verification state space by canonicalizing same-role agent permutations,
4. verify bounded safety and collect concrete unsafe evidence,
5. inject hard constraints from evidence into planner,
6. rerun until safe or iteration budget exhausted,
7. evaluate final behavior quantitatively.

This is the full execution pipeline your project conveys in practice.

23. Chapter Summary

This chapter is the runbook for operating and validating the full system.

It defines exact mode behavior, internal data artifacts, safety-loop transitions, experiment protocol, and interpretation guidelines so the project can be executed, analyzed, and reported consistently.